

El patrón de diseño mvc

M5: Desarrollo de aplicaciones web dinámicas en Java

|AE5: Aplicar patrón modelo-vista-controlador en una aplicación web dinámica para la implementación de un flujo de navegación que despliega y captura información de acuerdo a lo solicitado

Introducción

El desarrollo de aplicaciones web ha evolucionado adoptando patrones arquitectónicos que permiten crear sistemas escalables y mantenibles.

Uno de los más utilizados es el patrón **Modelo-Vista-Controlador** (MVC), que promueve la separación de conceptos y facilita el desarrollo de interfaces de usuario ricas.

En esta lección veremos en qué consiste MVC, sus elementos y cómo aplicar este patrón en el desarrollo de aplicaciones web dinámicas con Java EE. Aprenderás el papel del modelo, la vista y el controlador, así como los beneficios de utilizar MVC para crear sistemas flexibles y de fácil mantenimiento.

Desarrollaremos los conceptos necesarios para implementar el patrón MVC en tus desarrollos web, creando aplicaciones escalables y con una clara separación de responsabilidades por capas. ¡Comencemos!

Aprendizaje esperado

Al finalizar la lección podrás:

- Aplicar patrón modelo-vista-controlador en una aplicación web dinámica para la implementación de un flujo de navegación que despliega y captura información de acuerdo a lo solicitado.

¿Qué es un patrón de diseño?

Un patrón de diseño es una **solución reutilizable y general para un problema recurrente en el desarrollo de software**. Los patrones de diseño proveen **templates predefinidos para resolver situaciones comunes** en la construcción de aplicaciones, permitiendo resolver estos problemas de manera eficiente y probada.

Los patrones de diseño surgieron a partir de la necesidad de contar con soluciones estandarizadas para problemas habituales como creación de objetos, estructuración de clases e interfaces o relaciones entre componentes. En lugar de resolver estas cuestiones una y otra vez, los patrones capturan las mejores prácticas en formatos reutilizables.

Algunas características de los patrones de diseño:

- Son reutilizables en distintos contextos y lenguajes de programación.
- Resuelven problemas recurrentes en el desarrollo de software.
- Facilitan el uso de buenas prácticas probadas y estandarizadas.
- Se basan en principios de diseño orientado a objetos.
- Mejoran la legibilidad y mantenimiento del código.

En resumen, los patrones de diseño como MVC, Singleton, Observer y otros tantos, nos permiten resolver de forma óptima y reutilizable problemas comunes de diseño en nuestras aplicaciones.

Patrón MVC

MVC (Modelo-Vista-Controlador) es un patrón en el diseño de software comúnmente utilizado para implementar interfaces de usuario, datos y lógica de control. Enfatiza una **separación entre la lógica de negocios y su visualización**. Esta "separación de preocupaciones" proporciona una mejor división del trabajo y una mejora de mantenimiento.

Las tres partes del patrón de diseño de software MVC se pueden describir de la siguiente manera:

Modelo: Maneja datos y lógica de negocios.

Vista: Se encarga del diseño y presentación.

Controlador: Enruta comandos a los modelos y vistas.

Beneficios de utilizar MVC

Utilizar el patrón MVC en aplicaciones web aporta varios beneficios:

- Permite separar la lógica de negocio de la interfaz de usuario, resultando en una mejor organización del código.
- Promueve la reutilización de código, por ejemplo se puede usar el mismo modelo para distintas interfaces.
- Facilita el mantenimiento y actualización, ya que se pueden modificar el modelo, vista o controlador de forma independiente.
- El desarrollo se puede dividir por roles: programadores de backend en el modelo, diseñadores frontend en la vista, etc.
- Los cambios tienen menor impacto al estar desacoplados en capas separadas.
- Permite escalar cada componente de forma independiente.
- Es más fácil realizar pruebas unitarias enfocadas en cada capa.
- La interfaz visual queda abstracta del modelo de datos subyacente.
- Promueve un acoplamiento más débil entre los componentes.
- Siguiendo estándares resulta en un código más claro y mantenible.

Elementos que van en el Modelo

En una aplicación web dinámica desarrollada con Java, los elementos típicos que pertenecen a la capa de Modelo en el patrón MVC son:

- Clases POJO (**P**lain **O**ld **J**ava **O**bjects) que representen **entidades** y objetos de negocio. Por ejemplo, clases Usuario, Producto, Factura, etc.
- Objetos de acceso a datos, como DAOs (Data Access Objects), para abstraer la lógica de **persistencia**.
- Clases que implementen la lógica de negocio más allá de los datos, como cálculos, algoritmos, validaciones, etc.
- Objetos de dominio que modelen conceptos específicos del negocio.
- Conexiones a sistemas externos como bases de datos o APIs.
- Librerías y frameworks reusables enfocados en la capa de negocio.
- Interfaces y clases de **servicios** que abstraiga funcionalidades de negocio.

En resumen, el **modelo contiene toda la lógica de negocio y datos**, independiente de la visualización y la lógica de control. Solo se enfoca en encapsular las reglas y objetos de dominio.

Elementos que van en la Vista

En el patrón MVC, los elementos típicos que pertenecen a la capa de Vista en una aplicación web dinámica con Java son:

- Páginas **JSP** que definen la interfaz de usuario. Contienen HTML, CSS y código JSP para dinamismo.
- Archivos **estáticos** como imágenes, CSS y JavaScript utilizados para la presentación.
- Componentes visuales reusables como custom tags o macros.
- Layouts y templates para estructurar contenidos comunes.
- Librerías de etiquetas **JSTL** para facilitar la visualización de datos.
- Internacionalización mediante archivos de propiedades.
- Formularios para recolectar inputs del usuario.
- Clases que implementen funcionalidades de renderizado.

En resumen, la capa de **Vista** contiene todo lo relacionado a la **interfaz de usuario y la presentación visual** de la información, separado de la lógica de negocios.

Elementos que van en el Controlador

Los principales elementos que pertenecen a la capa de Controlador en aplicaciones web MVC con Java son:

- **Servlets** que implementan la lógica de control y manejo de requests y responses.
- **Filtros** (filters) para interceptar y modificar requests/responses.
- **Clases controladoras** Front Controller cuando se utiliza ese patrón.
- Acciones específicas para manejar lógica de control de ciertas peticiones.
- APIs de mapeo de URLs a métodos como en los frameworks MVC.
- Objetos que implementan workflow y navegación de la aplicación.
- Control de **sesión y autenticación** de usuarios.
- Transiciones y redireccionamiento según estado de la aplicación.
- Excepciones relacionadas al control de flujo.

El **controlador gestiona el flujo de la aplicación**, delegando trabajo al modelo y las vistas según corresponda.

Cierre

A lo largo de esta lección hemos visto en detalle en qué consiste el patrón Modelo-Vista-Controlador y cómo aplicarlo para el desarrollo de aplicaciones web dinámicas.

Estudiamos el importante concepto de los patrones de diseño como soluciones reutilizables a problemas comunes en el desarrollo de software.

Luego nos enfocamos en MVC, analizando cada una de sus capas y responsabilidades: el Modelo para los datos y lógica de negocio, la Vista para la interfaz de usuario, y el Controlador para la gestión del flujo de la aplicación. También vimos los múltiples beneficios que aporta este patrón, como el desacoplamiento, la reutilización de código y la independencia entre las capas.

¡Nos vemos de nuevo, más adelante! 

Referencias

- Refactoring.Guru. (s.f.). Patrones de diseño (Design Patterns). Recuperado el 28 de julio de 2025, de <https://refactoring.guru/es/design-patterns/>
- SourceMaking. (s.f.). Design Patterns. Recuperado el 28 de julio de 2025, de https://sourcemaking.com/design_patterns
- Blancarte, O. (21 de julio de 2014). Patrón de diseño Modelo-Vista-Controlador (MVC). Oscar Blancarte Blog. Recuperado el 28 de julio de 2025, de <https://www.oscarblancarteblog.com/2014/07/21/patron-de-diseno-modelo-vista-controlador-mvc/>
- CoDeJaVu. (2013, 17 de junio). Ejemplo Modelo Vista Controlador. Recuperado el 28 de julio de 2025, de <https://codejavu.blogspot.com/2013/06/ejemplo-modelo-vista-controlador.html>

¡Muchas gracias!

Nos vemos en la próxima lección

